

CS536 Science of Programming
Fall 2025
Assignment 6 Sample Solution Sketches

1. (a) True. Convergence is guaranteed by the existence of the bound expression t .
 (b) False. We need $p \Rightarrow t \geq 0$. Since p is still true after the last iteration of the loop, so $t \geq 0$ after the loop terminates.
 (c) True. Since the loop W is provable under total correctness, by the definition of the loop invariant and by the definition of the bound expression, we have $\vdash_{tot} \{p \wedge B \wedge t = t_0\} S \{p \wedge t < t_0\}$, which implies that $sp(p \wedge B \wedge t = t_0, S) \Rightarrow p \wedge t < t_0 \Rightarrow t < t_0$.
 (d) False. $p \wedge t > 0$ does not logically imply that B must be true; in other words, it is possible that a loop ends with $t > 0$.
2. (a) No. There is no evidence showing that $p \Rightarrow x - k + n \geq 0$.
 (b) No. There is no evidence showing that $p \Rightarrow n - k \geq 0$.
 (c) Yes. The loop invariant implies that $n - k + C \geq 0$; and k increases after each iteration (n and C are not updated in each iteration), so $n - k + C$ decreases after each iteration.
 (d) No. $k - C$ increases after each iteration.
 (e) Yes. $2^n \cdot 2^{(C-k)} = 2^{(n-k+C)}$, and the power in the expression is the same as the bound expression sub-question (c). Thus, $2^{(n-k+C)}$ is nonnegative and decreases after each iteration as well.
3. Since $p \Rightarrow y \geq r$ and r increases in each iteration, $y - r$ can be used as a bound expression. (There are other bound expressions as well.)

Here is the full proof outline under total correctness with the Backward Assignment Axiom used for all assignment statements.

$$\begin{aligned}
 &\{y \geq 1\} \{1 \leq 1 = 2^0 \leq y\} x := 0; \{1 \leq 1 = 2^x \leq y\} r := 1; \\
 &\{\mathbf{inv} \ p \equiv 1 \leq r = 2^x \leq y\} \{\mathbf{bd} \ y - r\} \\
 &\mathbf{while} \ 2 * r \leq y \ \mathbf{do} \\
 &\quad \{p \wedge 2 * r \leq y \wedge y - r = t_0\} \\
 &\quad \quad \{1 \leq 2 * r = 2^{(x+1)} \leq y \wedge y - 2 * r < t_0\} \\
 &\quad \quad \quad r := 2 * r; \{1 \leq r = 2^{(x+1)} \leq y \wedge y - r < t_0\} x := x + 1 \\
 &\quad \{1 \leq r = 2^x \leq y \wedge y - r < t_0\} \\
 &\mathbf{od} \\
 &\{p \wedge 2 * r > y\} \{r = 2^x \leq y < 2^{(x+1)}\}
 \end{aligned}$$

We need to prove the following logic implication to finish the proof:

- $y \geq 1 \Rightarrow 1 \leq 1 = 2^0 \leq y$, which is trivially true.
- $p \wedge 2 * r \leq y \Rightarrow 1 \leq 2 * r = 2^{(x+1)} \leq y$. Since $p \equiv 1 \leq r = 2^x \leq y$, so $1 \leq 2 * r = 2^{(x+1)}$. Since we are in the loop body, we also have $2 * r \leq y$.

- $y - r = t_0 \Rightarrow y - 2 * r < t_0$. This is true since $r \geq 1$.
 - $p \wedge 2 * r > y \Rightarrow r = 2^\wedge x \leq y < 2^\wedge(x + 1)$. This was proved in the previous assignment.
4. Here are the five possible loop-invariant candidates together with the corresponding loop conditions. I used u as a new variable to replace each constant.
- (a) $p_1 \equiv y \geq u \wedge x = 2 * y \leq n < 3 * (y + 1)$, and $B_1 \equiv u \neq 0$.
 - (b) $p_2 \equiv y \geq 0 \wedge x = u * y \leq n < 3 * (y + 1)$, and $B_2 \equiv u \neq 2$.
 - (c) $p_3 \equiv y \geq 0 \wedge x = 2 * y \leq u < 3 * (y + 1)$, and $B_3 \equiv u \neq n$.
 - (d) $p_4 \equiv y \geq 0 \wedge x = 2 * y \leq n < u * (y + 1)$, and $B_4 \equiv u \neq 3$.
 - (e) $p_5 \equiv y \geq 0 \wedge x = 2 * y \leq n < 3 * (y + u)$, and $B_5 \equiv u \neq 1$.
5. Here are the four possible loop-invariant candidates together with the corresponding loop conditions.
- (a) $p_1 \equiv (z = 2^\wedge y) \wedge (2^\wedge y \leq x) \wedge (x < 2^\wedge(y + 1))$, and $B_1 \equiv y < 0$.
 - (b) $p_2 \equiv (y \geq 0) \wedge (2^\wedge y \leq x) \wedge (x < 2^\wedge(y + 1))$, and $B_2 \equiv z \neq 2^\wedge y$.
 - (c) $p_3 \equiv (y \geq 0) \wedge (z = 2^\wedge y) \wedge (x < 2^\wedge(y + 1))$, and $B_3 \equiv 2^\wedge y > x$.
 - (d) $p_4 \equiv (y \geq 0) \wedge (z = 2^\wedge y) \wedge (2^\wedge y \leq x)$, and $B_4 \equiv x \geq 2^\wedge(y + 1)$.
6. (a) A reasonable precondition can be $n \geq 0$, which is inferred from the domain of function $fac(n)$. A reasonable post-condition can be $x = fac(n)$, it is given in the question.
- (b) A reasonable loop condition suggested in the question is $p \equiv x = fac(y) \wedge 0 \leq y \leq n$. Since the program will end with $y = n$, the loop condition should be $B \equiv y \neq n$.
- (c) We run the loop starting with $y = 0$ (since this is the other boundary of the range of y) and increase the value of y in each iteration, which means that we can include “ $-y$ ” in the bound expression. And $n - y$ is a good bound expression, since the loop invariant implies that $n - y \geq 0$.
- (d) Here is a possible full proof outline for this question.

```

{ $n \geq 0$ }
 $x := 1; \{n \geq 0 \wedge x = 1\} \ y := 0; \{n \geq 0 \wedge x = 1 \wedge y = 0\}$ 
{inv  $p \equiv x = fac(y) \wedge 0 \leq y \leq n\}$  {bd  $n - y$ }
while  $y \neq n$  do
    { $x = fac(y) \wedge 0 \leq y \leq n \wedge y \neq n \wedge n - y = t_0$ }
    { $x * (y + 1) = fac(y + 1) \wedge 0 \leq (y + 1) \leq n \wedge n - (y + 1) < t_0$ }  $x := x * (y + 1);$ 
    { $x = fac(y + 1) \wedge 0 \leq (y + 1) \leq n \wedge n - (y + 1) < t_0$ }  $y := y + 1$ 
    { $x = fac(y) \wedge 0 \leq y \leq n \wedge n - y < t_0$ }
od
{ $x = fac(y) \wedge 0 \leq y \leq n \wedge y = n$ }
{ $x = fac(n)$ }

```

There are other ways to write this program. For example, you might write the base case and the loop in an *if-else* statement; you might give x and y different initial values; your loop body might be different...

7. We can find p using the Backward Assignment Axiom, and we can create the following full proof outline.

$$\{p\} \ b[i] := b[j]; \{p_1\} \ b[j] := b[k] \ \{b[i] > b[k]\}$$

Here,

$$\begin{aligned} p_1 &\equiv (b[i] > b[k]) [b[k]/b[j]] \\ &\equiv (\text{if } i = j \text{ then } b[k] \text{ else } b[i] \text{ fi}) > (\text{if } k = j \text{ then } b[k] \text{ else } b[k] \text{ fi}) \\ &\mapsto (\text{if } i = j \text{ then } b[k] \text{ else } b[i] \text{ fi}) > b[k] \\ &\mapsto \text{if } i = j \text{ then } b[k] > b[k] \text{ else } b[i] > b[k] \text{ fi} \\ &\mapsto \text{if } i = j \text{ then } F \text{ else } b[i] > b[k] \text{ fi} \\ &\mapsto i \neq j \wedge b[i] > b[k] \end{aligned}$$

and,

$$\begin{aligned} p &\equiv (i \neq j \wedge b[i] > b[k]) [b[j]/b[i]] \\ &\equiv i \neq j \wedge b[j] > (\text{if } k = i \text{ then } b[j] \text{ else } b[k] \text{ fi}) \\ &\mapsto i \neq j \wedge (\text{if } k = i \text{ then } b[j] > b[j] \text{ else } b[j] > b[k] \text{ fi}) \\ &\mapsto i \neq j \wedge (\text{if } k = i \text{ then } F \text{ else } b[j] > b[k] \text{ fi}) \\ &\mapsto i \neq j \wedge k \neq i \wedge b[j] > b[k] \end{aligned}$$

8. We can use the Backward Assignment Axiom to create the following full proof outline.

$$\{k < b[k] < b[j]\} \{p\} \ b[b[k]] := b[j] \ \{b[k] \neq b[j]\}$$

Here,

$$\begin{aligned} p &\equiv (b[k] \neq b[j]) \ [b[j]/b[b[k]]] \\ &\equiv (\text{if } k = b[k] \text{ then } b[j] \text{ else } b[k] \text{ fi}) \neq (\text{if } j = b[k] \text{ then } b[j] \text{ else } b[j] \text{ fi}) \\ &\mapsto (\text{if } k = b[k] \text{ then } b[j] \text{ else } b[k] \text{ fi}) \neq b[j] \\ &\mapsto \text{if } k = b[k] \text{ then } b[j] \neq b[j] \text{ else } b[k] \neq b[j] \text{ fi} \\ &\mapsto \text{if } k = b[k] \text{ then } F \text{ else } b[k] \neq b[j] \text{ fi} \\ &\mapsto k \neq b[k] \wedge b[k] \neq b[j] \end{aligned}$$

It is easy to see that $k < b[k] < b[j] \Rightarrow k \neq b[k] \wedge b[k] \neq b[j]$, and we finish the proof outline.